

Temperature Dependence of a Diode

Junyu Hu,^{*} Songhang Li,[†] and Kowalski Sai[‡]
Franklin & Marshall College Department of Physics
(Dated: March 14, 2026)

Diodes are widely used in the electronic era. The conductivity of semiconductive material in the PN junction is strongly dependent on the temperature due to the minor current density generated by depletion region electric field. In this experiment, we measured the IV curve of a diode over a temperature range of 4K to room temperature. We determined the thermally-generated minority current I_s each temperature and then determined the band gap energy E_{gap} of the semiconductor by fitting I_s as a function of temperature. We also discussed why the IV curve fitting doesn't work when the temperature is below the 40 K.

1. INTRODUCTION

The diode is a ground-breaking invention that has a profound impact on modern society. A diode is an electronic component that permits flow of current in one direction. It has low resistance in one direction and high resistance in the other. The mechanism behind that is the movement of electrons in the PN junction. This one-directional behavior is essential for conversion from AC to DC in power supplies and protecting circuits from voltage spikes. Modern applications such as light-emitting diodes (LEDs), lasers, and solar cells, also show how this simple structure impacts today's technology.

2. THEORY [1]

A diode has a PN junction, formed by joining the p-type and n-type semiconductive material. In a p-type material, holes (positive charge carriers) are the majority, while electrons (negative charge carriers) are the majority in the n-type materials. At thermal equilibrium, carrier diffusion across the junction creates a depletion region that produces an internal electric field E , opposing further diffusion. This equilibrium therefore creates a potential across the junction.

Under forward bias (positive voltage applied to the p-side), the external electric field overcomes the barrier, enabling majority carriers to flow through. The resulting current I depends on the applied voltage V and temperature T .

In a PN junction diode, the current densities J_{nr} , J_{pr} , J_{ng} , and J_{pg} describe the contributions of majority and minority carriers to the total current:

$$\begin{aligned} J_{nr} &= J_{ng} e^{q|V|/k_B T} \\ J_{pr} &= J_{pg} e^{q|V|/k_B T} \end{aligned} \tag{1}$$

,where

- J_{nr} = *Majority electron current density*: dominates under forward bias and describes the flow of electrons from the **n**-side (where they are majority carriers) to the p-side. These electrons recombine with holes in the p-side.
- J_{pr} = *Majority hole current density*: represents the flow of holes from the p-side (where they are majority carriers) to the n-side. Holes recombine with electrons in the n-side.
- J_{ng} = *Minority electron current density*: formed by thermally generated electrons on the p-side (minority carriers). These electrons are drawn back to the n-side by the built-in electric field in the depletion region. It depends solely on temperature, instead of applied voltage.

^{*}Electronic address: jhu2@fandm.edu

[†]Electronic address: sl2@fandm.edu

[‡]Electronic address: ysai@fandm.edu

- J_{pg} = *Minority hole current density*: formed by thermally generated holes on the n-side (minority carriers). These holes are swept back to the p-side by the built-in electric field. Independent of applied voltage.

At thermal equilibrium (no bias voltage):

$$J_{nr} = J_{ng} \quad \text{and} \quad J_{pr} = J_{pg}$$

resulting in zero net current.

Under forward bias:

$$J_{nr} = J_{ng}e^{qV/k_BT}, \quad J_{pr} = J_{pg}e^{qV/k_BT}$$

, where the majority currents (J_{nr}, J_{pr}) dominate because it is exponential. The exponential relationship between the majority carrier currents and the forward bias voltage means that these currents increase significantly when applied voltage increases, overcoming the much smaller minority carrier generated currents.

The diode's total current I comes from the sum of recombination currents (forward bias-driven) and generation currents (thermally generated), scaled by a constant A :

$$I = A[(J_{nr} + J_{pr}) - (J_{ng} + J_{pg})]$$

Under forward bias V , recombination currents dominate and follow $J_{pr} = J_{pg}e^{qV/k_BT}$ and $J_{nr} = J_{ng}e^{qV/k_BT}$. By substituting these terms, we have:

$$I = A(J_{ng} + J_{pg})\left(e^{qV/k_BT} - 1\right)$$

This simplifies to the Shockley equation:

$$I = I_s\left(e^{q|V|/k_BT} - 1\right) \quad (2)$$

, where k_B is Boltzmann's constant, and q is the fundamental charge, $I_s = A(J_{ng} + J_{pg})$ is the saturation current, and E_{gap} is the semiconductor's band-gap energy. Notice that I_s is strongly temperature-dependent due to its relationship to thermal carrier generation, as detailed for E_{gap} in Eq 3.

These terms collectively define the diode's IV curve and its temperature dependence.

$$I_s \propto e^{-E_{\text{gap}}/k_BT} \quad (3)$$

By measuring IV curves at multiple temperatures, $I_s(T)$ can be extracted via fitting to Eq.3. Plotting $\ln(I_s)$ versus $1/T$ yields a linear relationship with slope $-E_{\text{gap}}/k_B$, enabling E_{gap} determination.

3. EXPERIMENTAL METHODS

We measured the range from 4K to room temperature with 20K increment. Total 14 sets of data.

The experimental setup consisted of a diode mounted on a closed cycle refrigerator(CCR, Janis Research SRDK-205) to vary its temperature between 4 K and room temperature. The CCR system included a vacuum chamber to prevent condensation, a helium circulation pump for cooling, and a Lake Shore 355 temperature controller with a resistive heating element for precise temperature control. The diode was connected to custom LabVIEW interface that applied forward bias voltage, and measuring the voltage and current across the diode using Agilent 34401A multimeter.

To characterize the diode's temperature-dependent behavior, we recorded IV curves at multiple temperatures. The diode was first cooled down to 4 K. The vacuum chamber was evacuated to pressure ≤ 1 mTorr using a pumping station to minimize condensation. After the temperature stabilizes, the LabVIEW program adjusts the voltage from -2.5 V to +2.5 V in 0.1 increments, recording 50 data points per IV curve measurement.

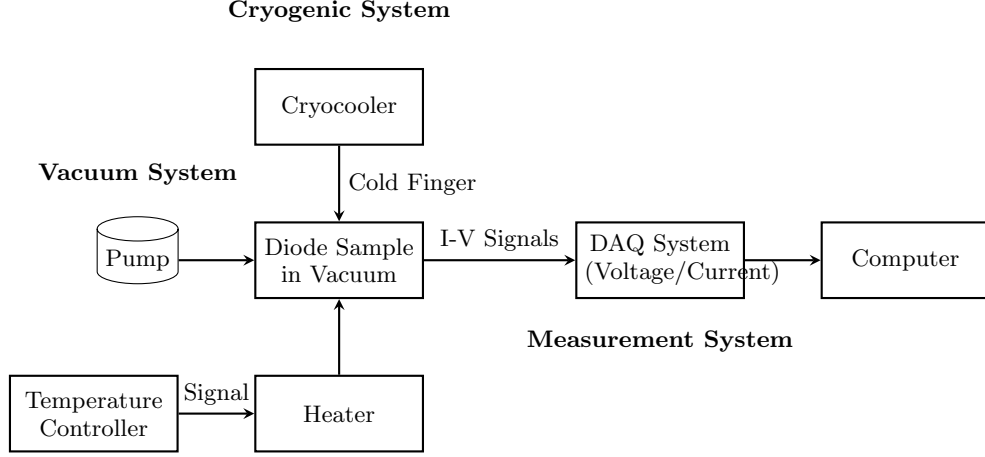


FIG. 1: Schematic diagram of the experimental setup for diode characterization. The diode is mounted in the closed-cycle refrigerator (CCR) vacuum chamber, cooled via helium circulation. A forward bias voltage is applied, the voltage across (V) and current through (I) the diode are measured by 2 Agilent multimeters respectively. The Lake Shore controller managed the temperature T , and data is recorded via LabVIEW. Minority currents (J_{ng} , J_{pg}) depends on temperature, while majority currents (J_{nr} , J_{pr}) dominate when forward bias.

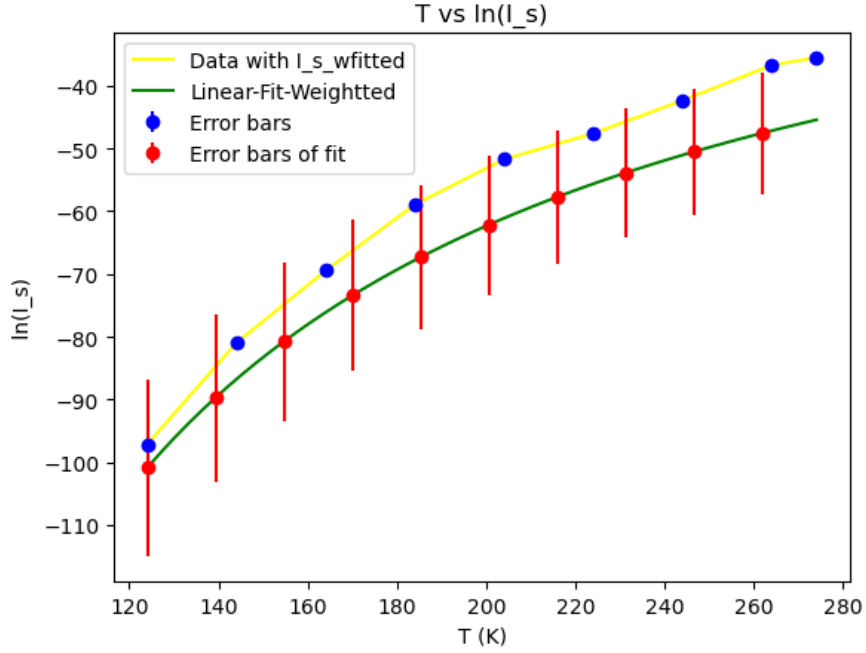


FIG. 2: The linear fit based on our model, where inputted I_s is from weighted curve fit we did

4. RESULTS AND ANALYSIS

4.1. Extraction of thermally-generated minority current(I_s)

For each temperature IV curve ($T \geq 40K$), Shockley equation(Eq. 2) is used for non-linear fitting. The forward bias zone ($V > 0.001V$) is selected to avoid noise interference and the non-ideal effects of the reverse bias zone. Then using Python package `SciPy.optimize.curve_fit` do the curvefit. Where the package requires a initial guess, the initial guess value is the mean value of I_s calculated by Eq. 2 within the same temperature.

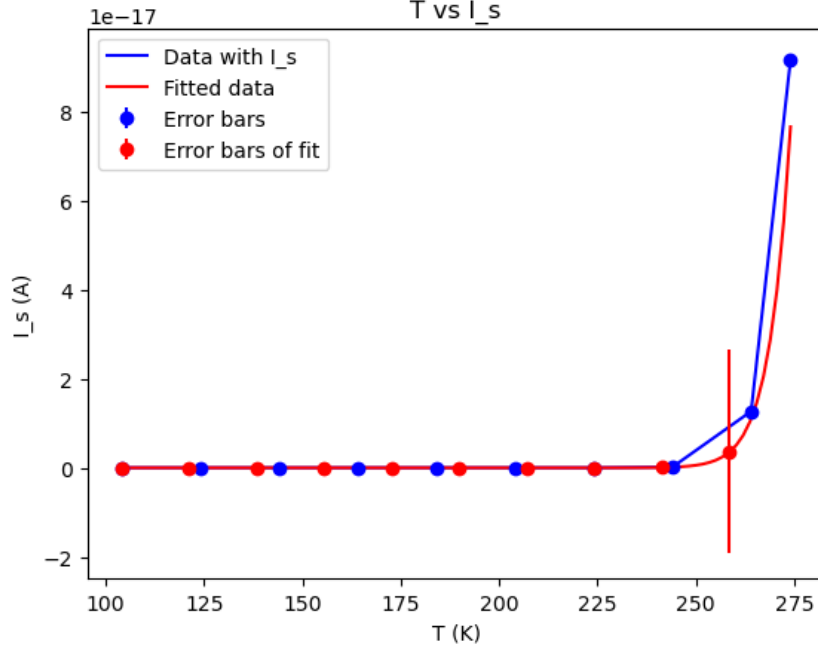


FIG. 3: The curve fit with original model (Eq.9) with initial gauss from linear fit

Notice that $T \leq 40K$ were excluded in the model fit because of the "freeze-out" of the dopants. In semiconductors, carriers are supplied by ionized impurities. When the dopant atoms freeze-out, carrier are frozen into their bound states, significantly reducing the majority carrier. [2] This effect makes the semiconductor behave more like an insulator, where the saturation current equation of semiconductor doesn't apply any more.

4.2. Linearization and regression analysis

We fit the value of band-gap (E_{gap}) using the package of linear regression for it is hard to provide the initial guess with the same method as we estimated I_s (thermally-generated minority current). From Eq. 2, we assume the relationship between I_s and E_{gap} as follows:

$$I_s = Ce^{-E_{gap}/k_bT} \quad (4)$$

where C is also an unknown variable need to be fitted. Due to the limitation of python package, we need to provide two initial values for estimation of E_{gap} and C in Eq. 4. To estimate E_{gap} , we decided to use linearization method. From Eq. 4, we can get:

$$\ln(I_s) = \ln C - \frac{E_{gap}}{k_B} \cdot \frac{1}{T} \quad (5)$$

the experimental data is linearized as follows:

- Horizontal axis: $x = \frac{1}{T} (K^{-1})$
- Vertical axis: $y = \ln(I_s)$
- Do the linear fit: $y = kx + b$

In this step, we use the function **linregress** in Python package **scipy.stats**. Then uncertainty of the slope k is calculated as:

$$\Delta k = \sqrt{\frac{\sum (y_i - kx_i - C)^2}{(N-2) \sum (x_i - \bar{x})^2}}, \quad (6)$$

and the uncertainty of the band gap energy is:

$$E_{gap} = -k_B \cdot \Delta k \quad (7)$$

4.3. Higher order expansion and analysis

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n \quad (8)$$

Following with the method above, we continue to the higher order exponential model (derived by Taylor Expansion Eq.8). The modified exponential model with polynomial corrections in the exponent is given by:

$$I_s(T) = A \exp \left(-\frac{E_{gap}}{k_B T} + \sum_{n=2}^N C_{n-1} \left(\frac{1}{T} \right)^n \right) \quad (9)$$

where:

- A : Pre-exponential factor (the C in Eq. 5)
- E_{gap} : Bandgap energy
- $C_1, C_2, \dots, C_{N-1}$: Higher-order correction coefficients
- k_B : Boltzmann constant in eV
- N : Maximum correction order

Then we do the linear fit also due to the limitation of python package. Firstly, On the basis of the original log-linear model, add the reciprocal term of high order temperature:

$$\ln(I_s) = C_0 - \frac{E_{gap}}{k_B T} + \sum_{n=2}^N C_{n-1} \left(\frac{1}{T} \right)^n \quad (10)$$

where $C_0 = \ln A$ is calculated from A in Eq. 9.

Repeat the same steps as in the previous section. In this experiment, we obtain the fitting function and E_{gap} of the second order correction term. However, since the error of linear fitting results will be larger after bringing them back to the original model. Linear fitting compresses the high dynamic range of the original I_s . For example, if the predict error of $\ln I_s$ is 0.1, then the predict error of I_s is $e^{0.1} \approx 1.1$. In this case, in order to avoid error amplification caused by linear fitting, we take the obtained result as the initial value and then bring it back to the original model for fitting. The Fig. 3 is the figure of fit with the original model.

4.4. Uncertainty

In controlling the temperature, uncertainty may come from the actual temperature fluctuation that is on the scale of 0.01K. The temperature of each data collection is recorded only once in the first measurement, but the temperature at each IV measurement fluctuates with time. The uncertainty might also come from impurity of the material.

Impurities can change minority carrier generation rates and the band gap structure. These factors can change the measured values of I_s and, therefore, the band gap energy E_{gap} .

The fit uncertainty of each temperature's IV curve is attached in the Appendix .

5. CONCLUSIONS

The measured band-gap energy $E_{\text{gap}} = 1.163 \pm 0.012 \text{ eV}$ which is close to the literature value for silicon (Si) 1.12 eV . The discrepancy is actually outside the range, but the value itself is closest to the silicon when compared to other typical semiconductor materials, such as Ge, GaAs and GaSb. In the higher order expansion and analysis, we used higher order fits to find a better result, instead of only one exponential term.

[1] Amy Lytle, *Temperature dependence of a diode* (2025), Phy 422, Department of Physics and Astronomy, Franklin and Marshall College.

[2] M. S.M.SZE, *Semiconductor Devices Physics and Technology* (John Wiley & Sons John, 2012), chap. 1, p. 39, 3rd ed.

The general formula for error propagation is:

$$\delta_q = \sqrt{\left(\frac{\partial q}{\partial x}\right)^2 \sigma_x^2 + \left(\frac{\partial q}{\partial y}\right)^2 \sigma_y^2 + \dots + \left(\frac{\partial q}{\partial z}\right)^2 \sigma_z^2}$$

For the quotient and product cases, we can use the shortcut formula

If

$$q = \frac{x \times \dots \times z}{u \times \dots \times w}$$

then

$$\frac{\delta q}{|q|} = \sqrt{\left(\frac{\delta x}{x}\right)^2 + \dots + \left(\frac{\delta z}{z}\right)^2 + \left(\frac{\delta u}{u}\right)^2 + \dots + \left(\frac{\delta w}{w}\right)^2}$$

For Shockley equation (Eq.2) to get I_s :

$$\delta I_s = \sqrt{\left(\frac{\partial I_s}{\partial V}\right)^2 \sigma_V^2 + \left(\frac{\partial I_s}{\partial I}\right)^2 \sigma_I^2 + \left(\frac{\partial I_s}{\partial T}\right)^2 \sigma_T^2} \quad (11)$$

Similarly, for our estimated initial guess of I_s :

$$\delta \ln I_s = \sqrt{\left(\frac{\partial \ln I_s}{\partial C}\right)^2 \sigma_C^2 + \left(\frac{\partial \ln I_s}{\partial E}\right)^2 \sigma_E^2 + \left(\frac{\partial \ln I_s}{\partial T}\right)^2 \sigma_T^2} \quad (12)$$

$$\delta k = \sqrt{\left(\frac{\partial k}{\partial x}\right)^2 \sigma_x^2 + \left(\frac{\partial k}{\partial m}\right)^2 \sigma_m^2 + \left(\frac{\partial k}{\partial m}\right)^2 \sigma_m^2} \quad (13)$$

Appendix A: Table

Semiconductor Band Gaps

2* Material	Energy gap (eV)	
	0 K	300 K
Si	1.17	1.11
Ge	0.74	0.66
InSb	0.23	0.17
InAs	0.43	0.36
InP	1.42	1.27
GaP	2.32	2.25
GaAs	1.52	1.43
GaSb	0.81	0.68
CdSe	1.84	1.74
CdTe	1.61	1.44
ZnO	3.44	3.2
ZnS	3.91	3.6

Data from Kittel, C., Introduction to Solid State Physics, 6th Ed., New York:John Wiley, 1986, p. 185.

Voltage-Current Data at 44.00 K

V (V)	I (A)
0.0499364620	-0.0000000970
0.1000613400	-0.0000000746
0.1501075000	-0.0000000448
0.1998824200	-0.0000000373
0.2499379700	-0.0000000448
0.3000342700	-0.0000000075
0.3500829000	-0.0000000299
0.3998562100	0.0000000075
0.4499160100	0.0000000025
0.5000118100	-0.0000000896
0.5500650700	-0.0000000075
0.5998421000	0.0000000423
0.6498632000	0.0000000274
0.6999783500	0.0000000100
0.7500566600	-0.0000000323
0.8001088000	0.0000000373
0.8499065400	0.0000000100
0.8999657000	0.0000000697
0.9500260200	0.0000000846
1.0000899000	0.0000000323
1.0498644000	0.0000000871
1.0833142000	0.0000165040
1.0905174000	0.0000587291
1.0940965000	0.0001046934
1.0965724000	0.0001513716
1.0985363000	0.0001987838
1.1001928000	0.0002466688
1.1016431000	0.0002945785
1.1029423000	0.0003424211
1.1041378000	0.0003907338
1.1052491000	0.0004390343
1.1062907000	0.0004874192
1.1072723000	0.0005355180
1.1082116000	0.0005840522
1.1091107000	0.0006325294
1.1099764000	0.0006811183
1.1108050000	0.0007294609
1.1116126000	0.0007780174
1.1123973000	0.0008266511
1.1131636000	0.0008753720
1.1139044000	0.0009236773
1.1146336000	0.0009723881
1.1153461000	0.0010212009
1.1160453000	0.0010698572
1.1167256000	0.0011183018
1.1173997000	0.0011670400
1.1180616000	0.0012157930
1.1187165000	0.0012645464
1.1193578000	0.0013130905

Voltage-Current Data at 64.00 K

V (V)	I (A)
0.0499375530	-0.0000000522
0.1000574100	0.0000000174
0.1501047700	-0.0000000647
0.1998801800	-0.0000000796
0.2499718300	0.0000000000
0.3000697400	-0.0000000050
0.3500877300	-0.0000000348
0.3998604300	0.0000000124
0.4499230800	-0.0000000224
0.5000203700	0.0000000522
0.5500721400	-0.0000000373
0.5998484300	-0.0000000274
0.6498691500	-0.0000000025
0.6999814500	0.0000000025
0.7500601300	0.0000000274
0.8001115300	0.0000000348
0.8499103800	0.0000000522
0.8999790900	0.0000000075
0.9500267600	0.00000001020
1.0000512000	0.0000000697
1.0426451000	0.0000072370
1.0553458000	0.0000440835
1.0604878000	0.0000884282
1.0637016000	0.0001346910
1.0660439000	0.0001815111
1.0679148000	0.0002289857
1.0694765000	0.0002769128
1.0708201000	0.0003250191
1.0719996000	0.0003730233
1.0730640000	0.0004213785
1.0740343000	0.0004698207
1.0749265000	0.0005182478
1.0757508000	0.0005667024
1.0765262000	0.0006152340
1.0772556000	0.0006639227
1.0779462000	0.0007127106
1.0785972000	0.0007611851
1.0792217000	0.0008099680
1.0798202000	0.0008587984
1.0803951000	0.0009076560
1.0809450000	0.0009562100
1.0814787000	0.0010051198
1.0819955000	0.0010540125
1.0824956000	0.0011028402
1.0829779000	0.0011515560
1.0834499000	0.0012004832
1.0839098000	0.0012494353
1.0843579000	0.0012984723
1.0847929000	0.0013471060

Voltage-Current Data at 84.00 K

V (V)	I (A)
0.0499433890	-0.0000000398
0.1000619300	-0.0000000199
0.1501130800	-0.0000000697
0.1998873800	-0.0000000100
0.2499447900	0.0000000348
0.3000393500	0.0000000597
0.3500874900	-0.0000000398
0.3998598100	0.0000000050
0.4499208500	-0.0000000572
0.5000187600	0.0000000299
0.5500711500	-0.0000000398
0.5998489200	0.0000000274
0.6498700200	0.0000000498
0.6999835600	0.0000000323
0.7500649700	0.0000000622
0.8001143800	0.0000000274
0.8499160900	0.0000001343
0.8999716500	0.0000000299
0.9499801200	0.0000001194
0.9968224700	0.0000033088
1.0171433000	0.0000323835
1.0245162000	0.0000745439
1.0288355000	0.0001196648
1.0319025000	0.0001659600
1.0342808000	0.0002128325
1.0362479000	0.0002603093
1.0379238000	0.0003079678
1.0393863000	0.0003560392
1.0406820000	0.0004038544
1.0418565000	0.0004520405
1.0429301000	0.0005004303
1.0439180000	0.0005488152
1.0448306000	0.0005970359
1.0456890000	0.0006456748
1.0464942000	0.0006942612
1.0472558000	0.0007429397
1.0479722000	0.0007913519
1.0486578000	0.0008399684
1.0493127000	0.0008887812
1.0499403000	0.0009375940
1.0505379000	0.0009861157
1.0511167000	0.0010348837
1.0516740000	0.0010838037
1.0522134000	0.0011326339
1.0527307000	0.0011813149
1.0532360000	0.0012301774
1.0537259000	0.0012790900
1.0542015000	0.0013280769
1.0546621000	0.0013766561

Voltage-Current Data at 104.00 K

V (V)	I (A)
0.0499461150	-0.0000000100
0.1000642400	-0.0000000572
0.1501156900	-0.0000000025
0.1998889900	-0.0000000498
0.2499459100	-0.0000000224
0.3000410900	-0.0000000174
0.3500920700	-0.0000001095
0.3998611700	0.0000000274
0.4499204800	-0.0000000299
0.5000203700	0.0000000075
0.5500670500	0.0000000224
0.5998455800	0.0000000100
0.6498682800	0.0000000323
0.6999802100	0.0000000274
0.7500600100	0.0000000498
0.8001101600	0.0000000249
0.8499075300	0.0000000075
0.8999012000	0.0000001692
0.9480579100	0.0000020002
0.9758670100	0.0000240146
0.9860645000	0.0000630728
0.9917099500	0.0001068751
0.9955893500	0.0001524812
0.9985521900	0.0001988808
1.0009460000	0.0002456862
1.0029796000	0.0002930361
1.0047438000	0.0003407394
1.0063041000	0.0003885794
1.0076988000	0.0004363127
1.0089732000	0.0004844836
1.0101442000	0.0005327566
1.0112262000	0.0005811415
1.0122306000	0.0006292007
1.0131765000	0.0006776230
1.0140664000	0.0007261173
1.0149083000	0.0007748032
1.0157025000	0.0008230886
1.0164631000	0.0008716802
1.0171892000	0.0009203312
1.0178847000	0.0009691142
1.0185478000	0.0010175115
1.0191893000	0.0010663470
1.0198081000	0.0011151672
1.0204052000	0.0011639203
1.0209781000	0.0012125166
1.0215370000	0.0012612924
1.0220786000	0.0013102097
1.0226045000	0.0013590896
1.0231119000	0.0014077210

Voltage-Current Data at 124.00 K

V (V)	I (A)
0.0499495100	-8.4600e-8
0.1000688300	-1.2400e-8
0.1501197800	-5.9700e-8
0.1998919700	2.5000e-9
0.2499492600	-3.7300e-8
0.3000427000	5.0000e-9
0.3500955500	3.2300e-8
0.3998688600	-7.5000e-9
0.4499276700	1.7400e-8
0.5000252100	-3.4800e-8
0.5500766000	-6.7200e-8
0.5998527700	2.7400e-8
0.6498748600	8.7100e-8
0.6999864100	-5.0000e-9
0.7500655900	1.2400e-8
0.8001100400	3.9800e-8
0.8498281500	1.6420e-7
0.8984704000	1.5872e-6
0.9318774500	1.80115e-5
0.9455058800	5.39277e-5
0.9526687100	9.59314e-5
0.9574788400	1.40602e-4
0.9610899600	1.864319e-4
0.9639861800	2.330209e-4
0.9663990700	2.797514e-4
0.9684904600	3.271212e-4
0.9703315500	3.747075e-4
0.9719775500	4.223587e-4
0.9734605700	4.700819e-4
0.9748249000	5.180662e-4
0.9760854300	5.662571e-4
0.9772547800	6.144728e-4
0.9783432600	6.625694e-4
0.9793712200	7.108871e-4
0.9803408800	7.593217e-4
0.9812609400	8.078186e-4
0.9821284000	8.560420e-4
0.9829603900	9.045338e-4
0.9837554200	9.531675e-4
0.9845185700	1.001906e-3
0.9852460100	1.050231e-3
0.9859507600	1.098987e-3
0.9866295800	1.147620e-3
0.9872859400	1.196381e-3
0.9879157600	1.244895e-3
0.9885300800	1.293654e-3
0.9891251800	1.342521e-3
0.9897031600	1.391394e-3
0.9902619100	1.439853e-3

Voltage-Current Data at 144.00 K

V (V)	I (A)
0.0499387050	-0.0000000697
0.1000554600	0.0000000597
0.1501044000	-0.0000000274
0.1998795600	-0.0000000149
0.2499382200	-0.0000000050
0.3000372400	-0.0000000075
0.3500842600	0.0000000075
0.3998596800	-0.0000000299
0.4499196100	-0.0000000025
0.5000180100	0.0000000274
0.5500695300	0.0000000050
0.5998465700	0.0000000896
0.6498681600	0.0000000299
0.6999784700	0.0000000398
0.7500476000	0.0000000697
0.8000013900	0.0000000871
0.8485970000	0.0000014180
0.8858805100	0.0000139017
0.9030912600	0.0000463474
0.9120267300	0.0000869206
0.9178365300	0.0001303076
0.9221614500	0.0001753986
0.9255957200	0.0002214203
0.9284482800	0.0002680041
0.9308816100	0.0003146898
0.9330250900	0.0003619676
0.9349372800	0.0004094619
0.9366626500	0.0004571628
0.9382303900	0.0005047367
0.9396807900	0.0005526812
0.9410275000	0.0006006905
0.9422825600	0.0006489239
0.9434541400	0.0006968162
0.9445646200	0.0007451389
0.9456142800	0.0007935113
0.9466122200	0.0008418765
0.9475554700	0.0008900425
0.9484617500	0.0009385965
0.9493292200	0.0009870187
0.9501629400	0.0010356449
0.9509574800	0.0010839754
0.9517286900	0.0011325344
0.9524720000	0.0011811805
0.9531916200	0.0012298863
0.9538821000	0.0012782641
0.9545563300	0.0013269923
0.9552098400	0.0013757430
0.9558448700	0.0014245459
0.9564575800	0.0014730477

Voltage-Current Data at 164.00 K

V (V)	I (A)
0.0499427570	-0.0000000672
0.1000610300	-0.0000000124
0.1501118400	-0.0000000149
0.1998858900	-0.0000000672
0.2499419400	-0.0000000224
0.3000399700	-0.0000000323
0.3500877300	-0.0000000149
0.3998606700	0.0000000348
0.4499227100	-0.0000000075
0.5000191300	-0.0000000075
0.5500720100	-0.0000000323
0.5998453300	0.0000000299
0.6498641900	0.0000000448
0.6999604900	0.0000000050
0.7499243200	0.0000001493
0.7988668900	0.0000012787
0.8385376900	0.0000112473
0.8592597300	0.0000402175
0.8700766200	0.0000789000
0.8770382800	0.0001213988
0.8821036900	0.0001655495
0.8861146800	0.0002109094
0.8894295000	0.0002570503
0.8922546500	0.0003036241
0.8947092000	0.0003502601
0.8969027900	0.0003975305
0.8988796900	0.0004450050
0.9006789900	0.0004925714
0.9023250100	0.0005400831
0.9038565300	0.0005879356
0.9052847300	0.0006359203
0.9066209000	0.0006839369
0.9078728500	0.0007317720
0.9090624200	0.0007800698
0.9101903500	0.0008282582
0.9112640700	0.0008766060
0.9122812500	0.0009247147
0.9132595900	0.0009730697
0.9141980000	0.0010215591
0.9151003200	0.0010701035
0.9159625700	0.0011183690
0.9167992700	0.0011668434
0.9176072000	0.0012155094
0.9183883400	0.0012641036
0.9191399600	0.0013124412
0.9198738400	0.0013610724
0.9205855300	0.0014097461
0.9212781100	0.0014584917
0.9219462500	0.0015069263

Voltage-Current Data at 184.00 K

V (V)	I (A)
0.0499473050	-0.0000000050
0.1000656800	-0.0000000174
0.1501176700	-0.0000000299
0.1998911000	0.0000000373
0.2499496300	0.0000000473
0.3000443100	0.0000000075
0.3500915800	0.0000000000
0.3998644000	0.0000000050
0.4499234500	-0.0000000174
0.5000219800	0.0000000075
0.5500718900	0.0000000597
0.5998439600	0.0000000323
0.6498455900	0.0000000622
0.6998210800	0.0000001617
0.7488368300	0.0000012091
0.7903834700	0.0000097173
0.8141010900	0.0000353390
0.8268627700	0.0000721756
0.8350092900	0.0001135349
0.8409147300	0.0001570015
0.8455144900	0.0002015875
0.8493160000	0.0002472186
0.8525475000	0.0002933969
0.8553597500	0.0003400229
0.8578405900	0.0003866564
0.8600846700	0.0004337976
0.8621254600	0.0004811973
0.8639975600	0.0005287090
0.8657203300	0.0005761584
0.8673325900	0.0006238990
0.8688424000	0.0006717443
0.8702604300	0.0007197958
0.8715936300	0.0007676036
0.8728645600	0.0008156352
0.8740714900	0.0008638262
0.8752243500	0.0009120270
0.8763182900	0.0009601159
0.8773729100	0.0010083838
0.8783861100	0.0010568013
0.8793608600	0.0011052235
0.8802940600	0.0011532975
0.8812001300	0.0012018664
0.8820764000	0.0012503260
0.8829253900	0.0012988753
0.8837420000	0.0013471657
0.8845397500	0.0013958020
0.8853145700	0.0014443387
0.8860684200	0.0014931092
0.8867967200	0.0015413622

Voltage-Current Data at 204.00 K

V (V)	I (A)
0.0499423110	-0.0000000547
0.1000589400	-0.0000000174
0.1501060100	-0.0000000075
0.1998858900	-0.0000000124
0.2499420600	-0.0000000299
0.3000417100	0.0000000547
0.3500877300	-0.0000000448
0.3998609200	-0.0000000050
0.4499203500	-0.0000000124
0.5000141700	0.0000000348
0.5500580000	0.0000000124
0.5998057600	0.0000000547
0.6496806300	0.0000001940
0.6987381700	0.0000012514
0.7415088800	0.0000084634
0.7681085900	0.0000315998
0.7827416300	0.0000662273
0.7921232400	0.0001063950
0.7988825200	0.0001490928
0.8041443500	0.0001932510
0.8084303300	0.0002380885
0.8120813900	0.0002838015
0.8152524500	0.0003300319
0.8180571600	0.0003767152
0.8205628100	0.0004233837
0.8228507800	0.0004704899
0.8249499800	0.0005177353
0.8268875700	0.0005652271
0.8286809300	0.0006125746
0.8303671100	0.0006602781
0.8319532000	0.0007080485
0.8334471400	0.0007558687
0.8348574800	0.0008036764
0.8362044400	0.0008516784
0.8374875300	0.0008997622
0.8387154200	0.0009479705
0.8398831800	0.0009958255
0.8410103600	0.0010441233
0.8420946300	0.0010924363
0.8431409400	0.0011408063
0.8441429800	0.0011888777
0.8451174800	0.0012372502
0.8460604700	0.0012857199
0.8469747000	0.0013342441
0.8478560500	0.0013823755
0.8487176800	0.0014308947
0.8495538000	0.0014794040
0.8503691700	0.0015280202
0.8511571300	0.0015763233

Voltage-Current Data at 224.00 K

V (V)	I (A)
0.0499434380	-0.0000000796
0.1000613400	-0.0000000174
0.1501135800	-0.0000000124
0.1998863900	-0.0000000274
0.2499460300	0.0000000224
0.3000419600	0.0000000174
0.3500898400	-0.0000000323
0.3998636500	-0.0000001020
0.4499225800	0.0000000199
0.5000125600	0.0000000522
0.5500256300	0.0000000821
0.5996240600	0.0000002712
0.6485993400	0.0000013310
0.6922640800	0.0000076723
0.7213212900	0.0000283657
0.7379790000	0.0000612293
0.7485839300	0.0000998695
0.7562419000	0.0001417464
0.7621804400	0.0001852726
0.7670245700	0.0002297594
0.7710983200	0.0002749002
0.7746471800	0.0003207600
0.7777825500	0.0003669904
0.7805922000	0.0004135468
0.7831277300	0.0004602054
0.7854625900	0.0005072095
0.7876194800	0.0005545471
0.7896211900	0.0006018672
0.7914831200	0.0006490955
0.7932405000	0.0006967465
0.7948993900	0.0007444423
0.7964675000	0.0007922401
0.7979499400	0.0008398689
0.7993705800	0.0008878532
0.8007264700	0.0009358475
0.8020259300	0.0009839687
0.8032638600	0.0010318138
0.8044611200	0.0010800248
0.8056146000	0.0011282505
0.8067276400	0.0011765060
0.8077956700	0.0012244630
0.8088349100	0.0012728730
0.8098426500	0.0013212355
0.8108195100	0.0013696928
0.8117615100	0.0014177667
0.8126833000	0.0014661914
0.8135789200	0.0015147081
0.8144524600	0.0015632373
0.8152974700	0.0016114458

Voltage-Current Data at 244.00 K

V (V)	I (A)
0.0499444920	-0.0000000622
0.1000630000	-0.0000000149
0.1501137000	0.0000000075
0.1998878700	0.0000000050
0.2499472700	-0.0000000572
0.3000424500	0.0000000050
0.3500902100	-0.0000000448
0.3998588100	0.0000000398
0.4499091900	0.0000000249
0.4999614600	0.0000000821
0.5498130400	0.0000002712
0.5985322300	0.0000012837
0.6427140100	0.0000070454
0.6738279000	0.0000258132
0.6923846300	0.0000568881
0.7043248700	0.0000944586
0.7128475600	0.0001351140
0.7194778300	0.0001778790
0.7248727600	0.0002218756
0.7294216700	0.0002668423
0.7333389000	0.0003121025
0.7368120000	0.0003579324
0.7399226700	0.0004042873
0.7427409900	0.0004508936
0.7453058300	0.0004973604
0.7476852100	0.0005443944
0.7498961600	0.0005916673
0.7519596400	0.0006389504
0.7538860700	0.0006860963
0.7557120500	0.0007336030
0.7574408900	0.0007812889
0.7590803200	0.0008290269
0.7606341600	0.0008765985
0.7621272300	0.0009244137
0.7635545600	0.0009723781
0.7649257100	0.0010204545
0.7662338800	0.0010681431
0.7675005900	0.0011162494
0.7687231500	0.0011644054
0.7699047800	0.0012126336
0.7710399000	0.0012605859
0.7721451300	0.0013088240
0.7732178600	0.0013570770
0.7742585900	0.0014055143
0.7752636100	0.0014535086
0.7762476900	0.0015019085
0.7772049500	0.0015503232
0.7781382700	0.0015988651
0.7790415800	0.0016469614

Voltage-Current Data at 264.00 K

V (V)	I (A)
0.0499429180	-0.0000000896
0.1000605300	-0.0000000522
0.1501088700	-0.0000000672
0.1998816700	-0.0000000373
0.2499388400	-0.0000001194
0.3000336500	0.0000000547
0.3500780600	-0.0000000124
0.3998405800	-0.0000000398
0.4498522600	0.0000000846
0.4997303900	0.0000002861
0.5486830100	0.0000013658
0.5930202100	0.0000067170
0.6257055300	0.0000238603
0.6460313000	0.0000532162
0.6592496600	0.0000895477
0.6687263000	0.0001294767
0.6760134300	0.0001714455
0.6819666300	0.0002149596
0.6869797900	0.0002593316
0.6913102800	0.0003044774
0.6951082000	0.0003496954
0.6985238500	0.0003956273
0.7016178900	0.0004418804
0.7044455500	0.0004885338
0.7070391300	0.0005351001
0.7094605600	0.0005820047
0.7117219900	0.0006291808
0.7138428000	0.0006763716
0.7158318600	0.0007235847
0.7177236800	0.0007709770
0.7195206200	0.0008185459
0.7212292600	0.0008661697
0.7228534200	0.0009136913
0.7244167000	0.0009614717
0.7259147300	0.0010093889
0.7273560800	0.0010573785
0.7287335500	0.0011050394
0.7300699700	0.0011530263
0.7313613700	0.0012011102
0.7326119600	0.0012492960
0.7338134300	0.0012971389
0.7349865400	0.0013453023
0.7361245100	0.0013934959
0.7372312200	0.0014418260
0.7382994900	0.0014898153
0.7393466700	0.0015381206
0.7403663200	0.0015865083
0.7413610400	0.0016348509
0.7423251200	0.0016829572

Voltage-Current Data at 274.00 K

V (V)	I (A)
0.0499430540	-0.0000000920
0.1000596300	-0.0000000149
0.1501071300	-0.0000000124
0.1998801800	-0.0000000174
0.2499387200	0.0000000522
0.3000320400	0.0000000348
0.3500754500	-0.0000000050
0.3998228500	0.0000000398
0.4497749900	0.0000001816
0.4993786400	0.0000005896
0.5469515500	0.0000030699
0.5866295600	0.0000130310
0.6133901300	0.0000359509
0.6303412900	0.0000685907
0.6419659800	0.0001065468
0.6506427500	0.0001472867
0.6574975000	0.0001896113
0.6632043800	0.000233119
0.6680727900	0.0002779179
0.6723218100	0.0003230388
0.6760754500	0.0003684209
0.6794713800	0.0004144747
0.6825622200	0.0004607004
0.6853986600	0.0005072966
0.6880093600	0.0005537485
0.6904527400	0.0006006855
0.6927404700	0.0006478094
0.6948905300	0.0006950325
0.6969098500	0.0007420789
0.6988328100	0.0007895085
0.7006613800	0.0008371446
0.7024028800	0.0008846689
0.7040600700	0.0009321407
0.7056562000	0.0009800281
0.7071873500	0.0010277487
0.7086613200	0.0010756811
0.7100710400	0.0011233520
0.7114397000	0.0011712991
0.7127624800	0.0012193356
0.7140429800	0.0012674795
0.7152750900	0.0013153395
0.7164776800	0.0013635156
0.7176441800	0.0014116343
0.7187806600	0.0014598450
0.7198762200	0.0015078319
0.7209524200	0.0015561048
0.7219993600	0.0016044825
0.7230217300	0.0016528351
0.7240124900	0.0017008593

$$y_i \ln(I_s)$$

$$\Delta(\ln(I_s)) = \frac{\partial \ln(I_s)}{\partial z_s} \cdot \Delta z_s \quad \Delta \bar{E} = \frac{\partial \bar{E}}{\partial k} \cdot \Delta k$$

$$= \frac{\Delta I_s}{I_s} \quad = k_B \cdot k$$

$$\Delta y = \frac{\partial y}{\partial I_s} \cdot \Delta I_s$$

$$I_s = \frac{I}{e^{q|V|/k_B T} - 1}$$

$$I_s \propto e^{-E_{gap}/k_B T}$$

$$I_s \propto e^{-E_{gap}/k_B T} + C$$

$$\ln(I_s) = C - \left(\frac{E_{gap}}{k_B} \right) \cdot \left(\frac{1}{T} \right)$$

$$\ln I$$

$$I = f(V, T, I_s)$$

$$T \rightarrow$$

$$I_s$$

$$I_s [\dots]$$

$$\downarrow$$

$$T I$$

$$]$$

$$\rightarrow I_s = \text{func}(T, E, C)$$

$$\ln(I_s) = C - \boxed{E_{\text{gap}}/k_B} \cdot \frac{1}{T}$$

$$y = b + k \cdot x$$

$$C \leftarrow \text{ini_val}$$

$$\text{slope} = -E_{\text{gap}}/k_B$$

$$- \text{slope} \cdot k_B = E_{\text{gap}}$$

$$\partial E = \frac{d}{d} d\text{slope} = -k_B \cdot \text{slope}$$

$$I_s = \frac{I}{e^{qV/kBT} - 1}$$

Γ $\left[\begin{array}{c} \downarrow \\ \Gamma, v \\ \vdots \end{array} \right]$

$$I = f_{\text{net}}(V, I_0, \overline{I})$$

$$\begin{bmatrix} 1 & - & - & - & - \\ 1 & - & - & - & - \end{bmatrix}$$

$$I_s = e^{-T_{gap}/k_B T} \quad A \text{ or } C.$$

Is $\text{fun}(\text{Ego}, \text{C}, \text{T})$ ini_vals $\times$

$$\ln(I_s) = C \left[-\frac{E_{gap}}{k_B} \right] + \left[\frac{1}{T} \right] \rightarrow x$$

(Linear.)) = slope, intercept.

$$T_{\text{gap}} = -\text{slope} \cdot R_B$$

$$\text{std err} = \left(\frac{\partial \bar{E}}{\partial S} \right)^2 \left(\Delta \text{Slope} \right)^2$$

↳ $k_B \cdot \text{std}$

$$I_s(T) = A \cdot \exp \left(- \frac{E_{\text{gap}}}{k_B T} + \sum_{n=2}^N C_{n-1} \left(\frac{1}{T} \right)^n \right)$$

$$A = e^{C_0}$$

E_{gap} ...

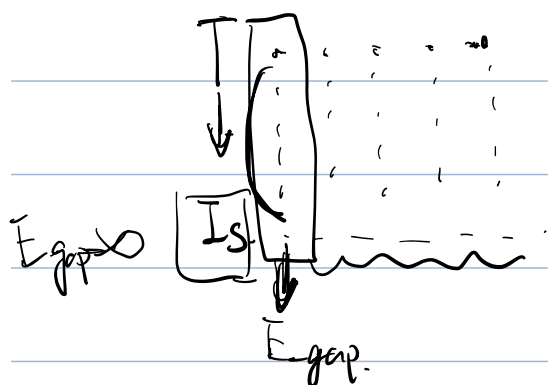
$C_1, C_2, \dots, C_{n-1} =$

k_B

N Maximum correction order.

$$I_s = A \exp \left[- \frac{E_{\text{gap}}}{k_B T} + C_1 \left(\frac{1}{T} \right)^2 + C_2 \left(\frac{1}{T} \right)^3 \right]$$

$$\ln(I_s) = C_0 - \frac{E_{\text{gap}}}{k_B} \cdot \frac{1}{T} + C_1 \left(\frac{1}{T} \right)^2 + C_2 \left(\frac{1}{T} \right)^3$$



$$\ln(I_{SD}) = C_0 - \frac{E_{gap}}{k_B} \cdot \frac{1}{T}$$

↑
↑ error

$$\Delta(\ln I_{SD}) = \sqrt{\frac{df^2}{dC_0} \Delta C_0^2 + \frac{df^2}{dT} \Delta T^2}$$

$$\sqrt{1 \cdot \Delta C_0^2 + -\frac{1}{k_B T} \cdot \Delta E_{gap}^2}$$

TABLE I: Fitted I_s Values and Associated Errors at Different Temperatures

T (K)	Type	I_s (A)	Error (A)	RMS Error	Error Rate
44	Unweighted	1.181×10^{-131}	9.600×10^{-133}	1.987×10^{-03}	8.126×10^{-02}
	Weighted	3.745×10^{-130}	1.834×10^{-130}	1.084×10^{-01}	4.898×10^{-01}
64	Unweighted	6.296×10^{-89}	2.147×10^{-90}	9.656×10^{-04}	3.411×10^{-02}
	Weighted	1.007×10^{-88}	3.333×10^{-89}	2.630×10^{-03}	3.312×10^{-01}
84	Unweighted	8.562×10^{-67}	2.123×10^{-68}	7.343×10^{-04}	2.479×10^{-02}
	Weighted	9.551×10^{-67}	2.370×10^{-67}	8.849×10^{-04}	2.482×10^{-01}
104	Unweighted	4.253×10^{-53}	8.673×10^{-55}	6.275×10^{-04}	2.039×10^{-02}
	Weighted	6.995×10^{-51}	1.339×10^{-51}	7.261×10^{-01}	1.914×10^{-01}
124	Unweighted	9.188×10^{-44}	1.622×10^{-45}	5.635×10^{-04}	1.765×10^{-02}
	Weighted	4.443×10^{-43}	7.230×10^{-44}	1.768×10^{-02}	1.627×10^{-01}
144	Unweighted	5.524×10^{-37}	8.929×10^{-39}	5.347×10^{-04}	1.616×10^{-02}
	Weighted	4.183×10^{-36}	5.653×10^{-37}	3.138×10^{-02}	1.351×10^{-01}
164	Unweighted	7.809×10^{-32}	1.193×10^{-33}	5.233×10^{-04}	1.528×10^{-02}
	Weighted	4.281×10^{-31}	4.905×10^{-32}	2.217×10^{-02}	1.146×10^{-01}
184	Unweighted	8.793×10^{-28}	1.323×10^{-29}	5.336×10^{-04}	1.505×10^{-02}
	Weighted	3.832×10^{-27}	3.821×10^{-28}	1.719×10^{-02}	9.972×10^{-02}
204	Unweighted	1.643×10^{-24}	2.465×10^{-26}	5.503×10^{-04}	1.500×10^{-02}
	Weighted	3.623×10^{-24}	3.108×10^{-25}	6.404×10^{-03}	8.578×10^{-02}
224	Unweighted	8.122×10^{-22}	1.223×10^{-23}	5.710×10^{-04}	1.506×10^{-02}
	Weighted	3.261×10^{-20}	2.652×10^{-21}	2.143×10^{-01}	8.133×10^{-02}
244	Unweighted	1.487×10^{-19}	2.278×10^{-21}	6.004×10^{-04}	1.532×10^{-02}
	Weighted	3.890×10^{-17}	2.845×10^{-18}	1.474×10^{00}	7.315×10^{-02}
264	Unweighted	1.267×10^{-17}	1.992×10^{-19}	6.366×10^{-04}	1.572×10^{-02}
	Weighted	1.161×10^{-18}	8.085×10^{-20}	5.347×10^{-03}	6.966×10^{-02}
274	Unweighted	9.158×10^{-17}	1.452×10^{-18}	6.522×10^{-04}	1.585×10^{-02}
	Weighted	6.424×10^{-15}	4.102×10^{-16}	4.106×10^{-01}	6.385×10^{-02}

Appendix C: source code

The Python code in three file for different function.

Here's the code in Main.py:

```
# This file is part of the Diode project.
# The Diode project is a simulation of a diode using the finite element method.
# the Main.py file is the data part of the project.

import numpy as np
import matplotlib.pyplot as plt
from Curvefit import *

def read_data(file_name):
    file_path = f"../Diode/Songhang/{file_name}"

    metadata = {}
    voltage = []
    current = []

    # open the file and read its contents
    with open(file_path, 'r') as file:
        lines = file.readlines()
```

```

# define a flag to check if we are in the metadata section
for line in lines:
    line = line.strip() # delete leading and trailing whitespace

    # if the line contains metadata, add it to the metadata list
    if "Diode DC Offset" in line or "Voltage Increment" in line or "Temperature" in line:
        key, value = line.split(":")
        numeric_value = float(value.strip().split()[0])
        metadata[key.strip()] = numeric_value

    # if the line contains data, split it and add it to the voltage and current lists
    elif line and not line.startswith("Voltage (V)": # skip the header line
        try:
            values = line.split() # split the line into values
            voltage.append(float(values[0])) # first column is Voltage
            current.append(float(values[1])) # second column is Current
        except ValueError:
            # overflow error, skip this line
            continue

return metadata, voltage, current

def plot_VvsI(voltage, current, I_s_unwfittd, I_s_wfittd, T):

    title = f"Fitted Diode Current at {T}K"

    x = np.linspace(min(voltage), max(voltage), 100)
    y_un = diode_current(x, I_s_unwfittd, T)
    y_w = diode_current(x, I_s_wfittd, T)

    # Plot the data
    plt.title(title)
    plt.plot(voltage, current, label="Data")
    plt.plot(x, y_un, label="Unweighttd-Fit", color='red')
    plt.plot(x, y_w, label="Weighttd-Fit", color='green')
    plt.xlabel("Voltage (V)")
    plt.ylabel("Current (A)")
    plt.legend()
    plt.show()

def plot_TvsIs(I_s_unw, E_gap_unwfittd, T, Cs, I_s_unwerr, perr, coefficients_err):

    title = f"T vs I_s"

    sigma_E_gap = perr
    sigma_coefficients = coefficients_err

    x = np.linspace(min(T), max(T), 100)
    y_un , sigma_result = propagated_uncertainty_exp(x,
                                                    E_gap_unwfittd,
                                                    Cs,
                                                    sigma_E_gap,
                                                    sigma_coefficients)

    n = int(len(x)/len(T))

    # Plot the data

```

```

plt.title(title)
plt.plot(T, I_s_unw, label="Data with I_s", color='blue')
plt.plot(x, y_un, label="Fitted data", color='red')

plt.errorbar(T, I_s_unw, yerr=I_s_unwerr, fmt='o', color='blue', label="Error bars")
plt.errorbar(x[:,n], y_un[:,n], yerr=sigma_result[:,n],
             fmt='o', color='red', label="Error bars of fit")

plt.xlabel("T (K)")
plt.ylabel(" I_s (A) ")
plt.legend()
plt.show()

def plot_HigherOrder(E_gap, I_s_w, T, coefficients, I_s_w_err, stderrors):

    title = f"T vs ln(I_s)"

    x = np.linspace(min(T), max(T), 50)
    sigma_E_gap = stderrors[1]
    sigma_coefficients = np.delete(stderrors, 1)
    y_fit, sigma_result = propagated_uncertainty(
        x, E_gap, *coefficients,
        sigma_E_gap=sigma_E_gap,
        sigma_coefficients=sigma_coefficients
    )
    # Plot the data
    plt.title(title)
    plt.plot(T, np.log(I_s_w), label="Data with I_s_wfitted", color='yellow')
    plt.plot(x, y_fit, label="Linear-Fit-Weighttted", color='green')

    n = int(len(x)/len(T))

    # Add error bars if provided
    plt.errorbar(T, np.log(I_s_w), yerr=I_s_w_err,
                 fmt='o', color='blue', label="Error bars")
    plt.errorbar(x[:,n], y_fit[:,n], yerr=sigma_result[:,n],
                 fmt='o', color='red', label="Error bars of fit")

    plt.xlabel("T (K)")
    plt.ylabel(" ln(I_s) ")
    plt.legend()
    plt.show()

def main():

    I_s_unw= []
    I_s_w = []
    Teps = []

    # this error saved for estimate the initial guess of E_gap
    I_s_werr = []
    I_s_unwerr = []

    for i in range(6,15):
        if i == 14:
            file_name = f"{274}k"

```

```

    metadata, voltage, current = read_data(file_name)
    Tep = 274

else:
    file_name = f"{4 + i*20}k"
    metadata, voltage, current = read_data(file_name)
    Tep = 4 + i*20

#remove the first data point
voltage = voltage[3:]
current = current[3:]

Teps.append(float(metadata["Temperature"]))

# estimate the initial guess for the fitting
exponent = q * np.abs(voltage) / (k * Tep)
exponent = np.clip(exponent, -700, 700)
I_s_estmited = np.mean(current/(np.exp(exponent) -1))

ini = [I_s_estmited]

# do unweighted fitting
I_s_unwfitted, perr, unweighted_rms = curvefit_unwI(voltage, current, Tep, ini)
I_s_unw.append(I_s_unwfitted)

I_s_unwerr.append(perr[0])
# do weighted fitting

I_s_wfitted, perr, weighted_rms = curvefit_wI(voltage, current, Tep, 0.01, 1e-10, ini)
if np.any(np.isnan(I_s_wfitted)):
    I_s_wfitted = I_s_unwfitted
I_s_w.append(I_s_wfitted)

# save the error for next fitting
I_s_werr.append(perr)

#estimate the initial guess for the fitting
# the curvefit need initial guess

##liner regression fit don't need initial guess

#sklearn.linear_model.LinearRegression don't need initial guess

Teps = np.array(Teps, dtype=float) # ensure Teps are float

I_s_w = np.array(I_s_w, dtype=float) # ensure I_s_w are float
I_s_werr = np.array(I_s_werr, dtype=float)

I_s_unw = np.array(I_s_unw, dtype=float)
I_s_unwerr = np.array(I_s_unwerr, dtype=float)

ln_I_swerr = I_s_werr / I_s_w
ln_I_sunerr = I_s_unwerr / I_s_unw

E_gap_h, coef, stderrors, p_value_F = liner_regression(Teps, I_s_w, ln_I_swerr, order=2)
print(f"HigherOrder Fitted E_gap: {E_gap_h :.3e} +- {stderrors[1] * k_eV :.3e} eV,
      p_value: {p_value_F:.3e}, order: {order}")

```

```

print(coef)
plot_HigherOrder(E_gap_h, I_s_w, Teps, coef, ln_I_swerr, stderrors)

#####

E_gap_unwfit, perr, coefficients, coefficients_err = curvefit_unwE_trf(Teps, I_s_unw)
print(f"Unweighted Fitted E_gap: {E_gap_unwfit:.3e} eV  error: {perr:.3e},
      error rate: {perr/E_gap_unwfit:.3e}")
print(coefficients)

plot_TvsIs(I_s_unw, E_gap_unwfit, Teps, coefficients, I_s_unwerr, perr, coefficients_err)

if __name__ == "__main__":
    main()

Here's the code in curfit.py:

# This file is the functional part of the project.
# we will use this file to define the mathematical functions that will be used in the project.

import numpy as np
from scipy.optimize import curve_fit
from scipy import odr
from scipy.stats import linregress
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from scipy.stats import f

# Constants
k = 1.380649e-23 # Boltzmann constant in J/K
k_eV = 8.617333262145e-5 # Boltzmann constant in eV/K
q = 1.602176634e-19 # Elementary charge in C

# eq(3) : I = I_s * (np.exp(q * np.abs(V)/kT) - 1)

# eq (4): np.(I_s) = C - E_gap/(kT)

def diode_current( V, I_s, T):
    exponent = q * np.abs(V) / (k * T)
    exponent = np.clip(exponent, -700, 700) # Limit the range of the exponent to avoid overflow
    return I_s * (np.exp(exponent) - 1)

def generated_current(T, E_gap, C):
    T = np.array(T, dtype=float) # Ensure T is a float array
    E_gap = float(E_gap) # Ensure E_gap is a float
    C = float(C) # Ensure C is a float
    return C - E_gap / (k_eV * T)

def generated_current_odr(T, E_gap, *coefficients):
    T = np.array(T, dtype=float)
    E_gap = float(E_gap)

    # Initialize the result, including the constant term C and the bandgap term
    result = coefficients[0] - E_gap / (k_eV * T) # C - E_gap / (k_eV * T)

    # Add higher-order terms
    for i, coeff in enumerate(coefficients[1:], start=2): # Start from T^2
        result += coeff / T**i

```

```

    return result

def propagated_uncertainty(T, E_gap, *coefficients, sigma_E_gap, sigma_coefficients):

    T = np.array(T, dtype=float)
    E_gap = float(E_gap)
    coefficients = np.array(coefficients, dtype=float)
    sigma_coefficients = np.array(sigma_coefficients, dtype=float)

    # Calculate result
    result = coefficients[0] - E_gap / (k_eV * T)
    for i, coeff in enumerate(coefficients[1:], start=2):
        result += coeff / T**i

    # Calculate uncertainty propagation
    sigma_result_squared = np.zeros_like(T)

    # Contribution from C
    sigma_result_squared += sigma_coefficients[0]**2

    # Contribution from E_gap
    partial_E_gap = -1 / (k_eV * T)
    sigma_result_squared += (partial_E_gap * sigma_E_gap)**2

    # Contribution from higher-order terms
    for i, coeff in enumerate(coefficients[1:], start=2):
        partial_coeff = 1 / T**i
        sigma_result_squared += (partial_coeff * sigma_coefficients[i-2])**2

    # Return result and uncertainty
    sigma_result = np.sqrt(sigma_result_squared)
    return result, 0.1 * np.sqrt(sigma_result)

# the function for curvefit with original function model
def generated_current_odr1(T, E_gap, *coefficients):
    T = np.array(T, dtype=float)
    E_gap = float(E_gap)
    result = coefficients[0] - E_gap / (k_eV * T)    # C0 - E_gap / (k_eV * T)

    for i, coeff in enumerate(coefficients[1:], start=2):    # Add higher-order terms
        result += coeff / T**i

    return np.exp(result)

def propagated_uncertainty_exp(T, E_gap, coefficients, sigma_E_gap, sigma_coefficients):
    """
    Calculate the uncertainty propagation of np.exp(result).

    Parameters:
    - T: Temperature array (numpy array)
    - E_gap: Bandgap value (float)
    - coefficients: Coefficient array (list or numpy array),
    including constant term C and higher-order terms
    - sigma_E_gap: Uncertainty of E_gap (float)
    - sigma_coefficients: Uncertainty array of coefficients
    (list or numpy array), corresponding to coefficients

    Returns:

```

```

- exp_result: Value of np.exp(result) (numpy array)
- sigma_exp_result: Uncertainty of np.exp(result) (numpy array)
"""
T = np.array(T, dtype=float)
coefficients = np.array(coefficients, dtype=float)
sigma_coefficients = np.array(sigma_coefficients, dtype=float)

# Boltzmann constant (eV/K)
k_eV = 8.617333262145e-5

# Calculate result
result = coefficients[0] - E_gap / (k_eV * T)
for i, coeff in enumerate(coefficients[1:], start=2):
    result += coeff / T**i

# Calculate uncertainty propagation of result
sigma_result_squared = np.zeros_like(T)

# Contribution from C
sigma_result_squared += sigma_coefficients[0]**2

# Contribution from E_gap
partial_E_gap = -1 / (k_eV * T)
sigma_result_squared += (partial_E_gap * sigma_E_gap)**2

# Contribution from higher-order terms
for i, coeff in enumerate(coefficients[1:], start=2):
    partial_coeff = 1 / T**i
    sigma_result_squared += (partial_coeff * sigma_coefficients[i-2])**2

# Calculate uncertainty of result
sigma_result = np.sqrt(sigma_result_squared)

# Calculate np.exp(result) and its uncertainty
exp_result = np.exp(result)
sigma_exp_result = exp_result * sigma_result

return exp_result, sigma_exp_result

def rms(pos, pos_fit):
    return np.sqrt(np.sum((pos-pos_fit)**2))

# the unweighted function of curve fitting
def curvefit_unwI(V, I, T, ini):

    # Input initial guesses. Make sure that the
    # order matches that of the function definition for the model.
    # [initP, initV, acc]
    init_vals = ini

    # Create the fit
    popt, pcov = curve_fit(lambda V, I_s: diode_current(V, I_s, T), V, I, p0=init_vals)

    # confirm the error in the fit
    perr = np.sqrt(np.diag(pcov))

    I_s_fitted = popt[0]

```

```

# Compute rms error:
unweighted_rms = rms(I, diode_current(V, I_s_fitted, T))

# Print fit values
# print(f'unWeighted fit: f_c = {popt[0]:.9f}')
```

Print value of rms error

```

# print(f'The error of the fit is {perr[0]:.9f}')
```

print(f'The rms error for the unweighted fit is {unweighted_rms:.3f}')

```

return I_s_fitted, perr, unweighted_rms

# the function of curve fitting with weighted error
def curvefit_wI(V, I, T, V_err, I_err, ini):

    # Input initial guesses. Make sure that
    # the order matches that of the function definition for the model.
    # [initP, initV, acc]
    init_vals = ini

    def diode_model(beta, V):
        I_s = beta[0]
        return diode_current(V, I_s, T)

    model = odr.Model(diode_model)

    # Create RealData object
    data = odr.RealData(V, I, sx=V_err, sy=I_err)

    # Setup odr with model and data
    myodr = odr.ODR(data, model, beta0=init_vals)

    # Run the regression
    out = myodr.run()

    I_s_fitted = out.beta[0]

    perr = np.sqrt(out.cov_beta[0][0])

    # Create a curve with the fitted parameters for visual comparison
    weighted_fit = diode_current(V, I_s_fitted, T)

    # Compute rms error:
    weighted_rms = rms(I, weighted_fit)

    # Print fit values
    # print(f'Weighted fit: f_c = {popt4[0]:.3f}')
```

print(f'The error of the fit is {error2[0]:.9f}')

Print value of rms error

```

# print(f'The rms error for the weighted fit is {weighted_rms:.3f}')
```

return I_s_fitted, perr, weighted_rms

```

def liner_regression(T, I_s, sigma_logI, order):
    # Calculate transformed variables
    InvT = 1 / T          # 1/T
    log_I_s = np.log(I_s) # ln(I_s)
    X = np.column_stack([InvT**i for i in range(1, order+1)])
```

```

X = np.column_stack([np.ones_like(InvT), X])

weights = 1 / sigma_logI**2
model = LinearRegression(fit_intercept=True)
model.fit(X, log_I_s, sample_weight=weights)

# Extract parameters
E_gap = -model.coef_[1] * k_eV # Restore E_gap from the first-order term coefficient
C0 = model.coef_[0]           # Intercept

X_weighted = X * np.sqrt(weights)[:, np.newaxis]
cov_matrix = np.linalg.inv(X_weighted.T @ X_weighted) # Covariance matrix
std_errors = np.sqrt(np.diag(cov_matrix)) # Standard deviation

# Calculate F-test
y_pred = model.predict(X) # Predicted values
# Regression sum of squares
SSR = np.sum(weights * (y_pred - np.average(log_I_s, weights=weights))**2)
# Residual sum of squares
SSE = np.sum(weights * (log_I_s - y_pred)**2)
n = len(log_I_s) # Number of samples
p = X.shape[1] - 1 # Degrees of freedom (number of regression coefficients - intercept)

# Calculate F-statistic
F_stat = (SSR / p) / (SSE / (n - p - 1))

# Calculate p-value
p_value_F = f.sf(F_stat, p, n - p - 1)

return E_gap, model.coef_, std_errors, p_value_F

# this fit don't need provide initial gauss
def curvefit_unwE_trf(T, I_s_unw, order=3):
    """
    Global optimization fitting for arbitrary order bandgap model
    Parameters:
        T (array): Temperature array (K)
        I_s_unw (array): Reverse saturation current array (A)
        order (int): Model order (default is 2nd order, i.e., C0 + C1/T^2)
    Returns:
        E_gap (float): Bandgap energy (eV)
        E_gap_err (float): Bandgap error (eV)
        coefficients (list): Correction term parameters [C0, C1, ...]
        coefficients_err (list): Correction term errors [C0_err, C1_err, ...]
    """
    # Check input validity
    T = np.asarray(T)
    I_s_unw = np.asarray(I_s_unw)
    assert len(T) == len(I_s_unw), "T and I_s_unw must have the same length"
    assert order >= 1, "Order must be 1"

    # Define wrapper function: use E_gap as the first parameter,
    # coefficients as subsequent parameters
    def _model_wrapper(T, E_gap, *coefficients):
        return generated_current_odr(T, E_gap, *coefficients)

    # Automatic parameter initialization strategy
    # -----

```

```

# Step 1: Estimate initial values of E_gap and C0
using logarithmic linear regression
log_I = np.log(I_s_unw)
X = np.column_stack([np.ones_like(T), 1/T])
coeffs_linear = np.linalg.lstsq(X, log_I, rcond=None)[0]

# Initial guess values
C0_guess = coeffs_linear[0] # Intercept term corresponds to C0
E_gap_guess = -coeffs_linear[1] * k_eV # Slope term corresponds to -E_gap/k_eV

# Initial guesses for higher-order terms set to 0
p0 = [E_gap_guess] + [C0_guess] + [0.0]*(order-1)

# Parameter boundary constraints
# -----
# Reasonable range for E_gap: 0.5 eV (Ge) ~ 2.0 eV (GaAs)
# Correction term coefficients allow positive
and negative fluctuations but limit magnitude
bounds_low = [0.5] + [-np.inf] + [-1e3]*(order-1)
bounds_high = [2.0] + [np.inf] + [1e3]*(order-1)
bounds = (bounds_low, bounds_high)

# Perform global optimization fitting
# -----
try:
    popt, pcov = curve_fit(
        _model_wrapper, T, log_I,
        p0=p0, bounds=bounds,
        method='trf', max_nfev=10000
    )
except RuntimeError as e:
    raise RuntimeError(f"Fitting failed: {e}")

# Parameter error estimation
perr = np.sqrt(np.diag(pcov)) if pcov is not None else [np.nan]*len(popt)

# Parse results
E_gap = popt[0]
E_gap_err = perr[0]
coefficients = list(popt[1:])
coefficients_err = list(perr[1:])

return E_gap, E_gap_err, coefficients, coefficients_err

if __name__ == "__main__":
    main()

```